

Red-Black Insertion Tracer

1. Introduction

A Red-Black Tree is a self-balancing Binary Search Tree (BST) that guarantees logarithmic time complexity for insertion and search operations. Unlike a standard BST, which may become skewed and degrade to $O(n)$ performance, a Red-Black Tree maintains balance through node coloring and rotation operations.

The purpose of this project is to implement a Red-Black Tree from scratch, support insertion operations, display the tree after every insertion, and verify that the tree satisfies the black-height property after each update.

2. Problem Statement

The task requires the implementation of a Red-Black Tree insertion tracer capable of:

- Inserting values into the tree.
- Automatically balancing the tree.
- Displaying node colors after each insertion.
- Verifying correctness through black-height validation.
- Providing a command-line demonstration application.
- Providing unit tests for correctness.

No deletion operation was required.

3. Data Structure Description

Binary Search Tree

A Binary Search Tree is a hierarchical data structure where:

- Values smaller than a node are stored in its left subtree.
- Values larger than a node are stored in its right subtree.

Although efficient in many cases, a standard BST can become unbalanced.

Red-Black Tree

A Red-Black Tree extends the BST by assigning each node a color.

Possible colors:

- RED
- BLACK

The following properties must always hold:

1. Every node is red or black.
2. The root is black.
3. Red nodes cannot have red children.
4. Every root-to-leaf path contains the same number of black nodes.
5. Null leaves are considered black.

These rules guarantee that the tree height remains logarithmic.

4. Design and Implementation

Node Structure

Each node contains:

```
int data;  
Color color;  
Node* left;  
Node* right;  
Node* parent;
```

The parent pointer simplifies tree rotations and balancing operations.

Insertion Process

Insertion occurs in two stages.

Stage 1: BST Insertion

The value is inserted using ordinary Binary Search Tree rules.

The new node is initially colored red.

Stage 2: Rebalancing

After insertion, violations of Red-Black Tree properties are corrected.

Possible actions include:

- Recoloring
- Left rotation
- Right rotation

The correction process continues until all properties are restored.

Tree Rotations

Left Rotation

A left rotation moves a node downward while promoting its right child.

Purpose:

- Restore balance when the right subtree becomes too heavy.

Complexity:

$O(1)$

Right Rotation

A right rotation moves a node downward while promoting its left child.

Purpose:

- Restore balance when the left subtree becomes too heavy.

Complexity:

$O(1)$

5. Correctness Verification

The project includes a black-height checker.

The checker recursively computes the black height of the left and right subtrees.

If:

`left_black_height == right_black_height`

the subtree is considered valid.

If any mismatch occurs:

0

is returned, indicating violation of the Red-Black Tree properties.

This verification is performed after every insertion.

6. Complexity Analysis

Insertion

A Red-Black Tree maintains logarithmic height.

Therefore:

Best Case: $\Omega(\log n)$

Average Case: $\Theta(\log n)$

Worst Case: $O(\log n)$

Search

The tree height is bounded by:

$2 \log_2(n + 1)$

Therefore:

Best Case: $\Omega(\log n)$

Average Case: $\Theta(\log n)$

Worst Case: $O(\log n)$

Rotation

Rotations involve a fixed number of pointer updates.

$O(1)$

Black Height Verification

Every node may be visited once.

$O(n)$

Space Complexity

One node is allocated for each inserted value.

$O(n)$

7. Testing

Unit tests were implemented using assert statements.

The tests verify:

- Basic insertion
- Multiple insertions
- Tree balancing
- Black-height validity

Example test sequence:

10, 20, 30, 15, 25, 5

After each insertion, the tree remains valid.

8. Results

The implementation successfully:

- Inserts elements into the tree.
- Maintains Red-Black Tree properties.
- Performs automatic balancing.
- Displays node colors.
- Verifies black-height correctness.
- Passes all unit tests.

The observed behavior matches the theoretical expectations of Red-Black Trees.

9. Conclusion

This project demonstrates the implementation of a Red-Black Tree using object-oriented C++ programming. Through insertion, rotations, recoloring, and validation, the tree remains balanced and guarantees efficient logarithmic performance.

The project illustrates how self-balancing trees overcome the limitations of ordinary Binary Search Trees and provides practical experience with advanced data structures commonly used in databases, operating systems, and search engines.